



南開大學
Nankai University

计算机学院
计算机系统设计实验报告

PA4 实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 5 月 27 日

目录

1 PA4 实验概述	2
2 PA4-1	2
2.1 在 NEMU 中实现分页机制	2
2.2 让用户程序运行在分页机制上	5
2.3 在分页机制上运行仙剑奇侠传	7
3 PA4-2	9
3.1 实现内核自陷	9
3.2 实现上下文切换	12
3.3 分时运行仙剑奇侠传和 hello 程序	14
4 PA4-3	17
4.1 添加时钟中断	18
5 展示你的计算机系统	21
5.1 实现思路	21
5.2 实现过程	22
5.3 代码解释说明	24
5.4 实现结果	25
6 思考题与必答题	26
6.1 思考题	26
6.2 必答题	31

1 PA4 实验概述

本次 PA4 实验围绕虚拟地址空间、分页机制、上下文切换、时钟中断和多任务展示展开。报告按阶段组织：第一阶段实现分页机制并让用户程序运行在虚拟地址空间；第二阶段实现内核自陷、上下文切换和基于系统调用的分时运行；第三阶段加入时钟中断，实现真正由硬件中断驱动的分时多任务；最后通过 F12 在仙剑奇侠传和 videotest 之间切换，展示完整系统能力。

- PA4-1：实现 i386 分页机制、用户程序分页加载、堆区动态映射。
- PA4-2：实现内核自陷、用户进程上下文创建与切换、PAL 和 hello 分时运行。
- PA4-3：实现时钟中断，让调度由硬件中断驱动。
- 展示系统：加载 videotest，并通过 F12 在两个图形程序之间切换。
- 思考题与必答题：分析分页、空指针、内核映射、中断嵌套和多任务运行机制。

2 PA4-1

本阶段围绕 i386 分页机制展开，目标是让 NEMU 支持分页地址转换，让 Nanos-lite 能把用户程序加载到独立虚拟地址空间，并在分页机制下支持用户程序动态扩展堆区，最终使 dummy 和仙剑奇侠传能够运行在分页机制之上。

2.1 在 NEMU 中实现分页机制

2.1.1 实现思路

i386 的分页机制通过 CR0 和 CR3 两个控制寄存器配合完成。CR0 的 PG 位用于表示分页是否开启，CR3 保存页目录的物理基地址。分页开启后，程序访问的地址不再直接作为物理地址，而是作为线性地址，经过页目录和页表两级转换后得到真正的物理地址。

因此，在 NEMU 中需要完成三件事：

第一，为 CPU 状态增加 CR0 和 CR3。

第二，实现访问 CR0 和 CR3 的指令，使 AM 中的 set_cr0()、set_cr3()、get_cr0() 能正常工作。

第三，在 vaddr_read() 和 vaddr_write() 中加入分页地址转换逻辑，使所有虚拟地址访存都经过 MMU 模拟。

本实验不实现完整的页级保护，只检查 PDE 和 PTE 的 present 位。如果访问了未映射页面，就直接触发 assertion，方便定位分页映射错误。同时，为了符合 i386 行为和 differential testing 的要求，在 page walk 过程中维护 accessed 位和 dirty 位。

2.1.2 实现过程

首先在 CPU_state 中增加 CR0 和 CR3：

```
1 EFLAGS eflags;
2 uint32_t cs;
3 CR0 cr0;
4 CR3 cr3;
```

CR0 和 CR3 的结构体定义已经在 `nemu/include/memory/mmu.h` 中给出, 因此只需要在 `reg.h` 中包含该头文件并把两个寄存器加入 `CPU_state`。

然后在 `restart()` 中初始化控制寄存器:

```
1 cpu.cr0.val = 0x60000011;
2 cpu.cr3.val = 0;
```

其中 `0x60000011` 是指导书要求的 CR0 初始值, PG 位此时为 0, 所以 NEMU 启动时分页并未开启。CR3 初始为 0, 后续由 AM 的 `_pte_init()` 通过 `set_cr3()` 设置。

接着实现控制寄存器访问指令。AM 中的内联汇编会生成如下两类指令:

```
1 movl %reg, %cr0
2 movl %reg, %cr3
3 movl %cr0, %reg
```

在 NEMU 中, 它们对应两字节 opcode:

```
1 0f 22 /r    mov r32 -> cr
2 0f 20 /r    mov cr -> r32
```

因此在 opcode 表中加入:

```
1 EX(mov_cr2r), EMPTY, EX(mov_r2cr), EMPTY
```

在 `mov_r2cr` 中解析 ModR/M 字节, 把通用寄存器的值写入 CR0 或 CR3:

```
1 switch (id_dest->reg) {
2     case 0: cpu.cr0.val = id_src->val; break;
3     case 3: cpu.cr3.val = id_src->val; break;
4     default: assert(0);
5 }
```

在 `mov_cr2r` 中则反过来, 把 CR0 或 CR3 的值写入通用寄存器:

```
1 switch (id_src->reg) {
2     case 0: id_src->val = cpu.cr0.val; break;
3     case 3: id_src->val = cpu.cr3.val; break;
4     default: assert(0);
5 }
6 rtl_sr_l(id_dest->reg, &id_src->val);
```

最后实现 `page_translate()`。i386 的 32 位线性地址被划分为 10 位页目录索引、10 位页表索引和 12 位页内偏移:

```
1 pdir_idx = addr >> 22;
2 ptab_idx = (addr >> 12) & 0x3ff;
3 page_off = addr & 0xfff;
```

地址转换伪代码如下:

```
1 if CR0.PG == 0:
```

```

2     return addr
3
4     pde_addr = CR3.page_directory_base + pdir_idx * sizeof(PDE)
5     pde = memory[pde_addr]
6     assert(pde.present)
7     set pde.accessed
8
9     pte_addr = pde.page_frame + ptab_idx * sizeof(PTE)
10    pte = memory[pte_addr]
11    assert(pte.present)
12    set pte.accessed
13    if this is write:
14        set pte.dirty
15
16    return pte.page_frame + page_off

```

具体实现中，CR3、PDE、PTE 中保存的都是页框号，需要左移 12 位得到 4KB 对齐的物理地址：

```

1 uint32_t pdir_base = cpu.cr3.page_directory_base << 12;
2 paddr_t pte_addr = (pde.page_frame << 12) + ptab_idx * sizeof(PTE);
3 return (pte.page_frame << 12) | page_off;

```

vaddr_read() 和 vaddr_write() 在分页关闭时仍然直接访问物理地址；分页开启时先调用 page_translate()。如果一次访问跨越页边界，则逐字节转换和访问：

```

1 if ((addr & PAGE_MASK) + len <= PAGE_SIZE) {
2     return paddr_read(page_translate(addr, false), len);
3 }
4
5 uint32_t data = 0;
6 for (int i = 0; i < len; i++) {
7     data |= paddr_read(page_translate(addr + i, false), 1) << (i * 8);
8 }
9 return data;

```

写内存时也采用相同思路：

```

1 if ((addr & PAGE_MASK) + len <= PAGE_SIZE) {
2     paddr_write(page_translate(addr, true), len, data);
3     return;
4 }
5
6 for (int i = 0; i < len; i++) {
7     paddr_write(page_translate(addr + i, true), 1, data >> (i * 8));
8 }

```

2.1.3 代码解释说明

CR0 和 CR3 是分页机制的硬件入口。AM 的 `_pte_init()` 会先建立内核页表，然后通过 `set_cr3()` 设置页目录地址，再通过 `set_cr0()` 打开 PG 位。NEMU 实现控制寄存器读写后，AM 的分页初始化才能真正作用到模拟器 CPU 状态。

`page_translate()` 模拟 i386 的 page walk。它不依赖虚拟地址访问页表，而是用 `paddr_read()` 和 `paddr_write()` 直接读取物理内存中的页目录项和页表项，因为 i386 手册规定 CR3、PDE、PTE 中保存的页表地址和物理页地址都是物理地址。

`present` 位用于检查页目录项和页表项是否有效。如果没有检查 `present` 位，错误映射可能继续产生不可预期的访存行为，使调试非常困难。

`accessed` 位在 PDE 和 PTE 被访问时置 1；`dirty` 位在写页面时置 1。这是 i386 分页硬件会自动维护的状态，NEMU 需要在软件模拟中补上。

2.2 让用户程序运行在分页机制上

2.2.1 实现思路

在 PA3 中，用户程序被直接加载到物理地址 0x4000000 并运行。开启分页后，用户程序应该运行在自己的虚拟地址空间中，而不是继续使用内核地址空间。指导书要求把 Navy-apps 的链接地址改为 0x8048000，使用户程序认为自己从该虚拟地址开始运行。Nanos-lite 的 loader 不再把程序内容直接读入虚拟地址，而是按页申请物理页，建立虚拟页到物理页的映射，再把文件内容读入对应物理页。

因此，这一要求的核心是实现 AM 的 `_map()`，并修改 `loader()` 的加载方式。

2.2.2 实现过程

首先在 Navy-apps 中修改用户程序链接地址：

```
1 LDFLAGS += -Ttext 0x8048000
```

这使用户程序内部的入口地址、函数地址和全局变量地址都落在 0x8048000 附近的虚拟地址空间中。

同时在 Nanos-lite 的 loader 中同步修改默认入口：

```
1 #define DEFAULT_ENTRY ((void *)0x8048000)
```

然后实现 `_map()`。`_map()` 的功能是把地址空间 `p` 中的虚拟页 `va` 映射到物理页 `pa`。由于本实现要求传入的 `va` 和 `pa` 都按页对齐，因此先检查页内偏移：

```
1 assert(OFF(va) == 0);
2 assert(OFF(pa) == 0);
```

接着根据虚拟地址计算页目录索引和页表索引：

```
1 uint32_t pdir_idx = PDX(va);
2 uint32_t ptab_idx = PTX(va);
```

如果对应 PDE 不存在，就申请一个新的页表并清零，然后把页表地址填入页目录项：

```
1 if ((pdir[pdir_idx] & PTE_P) == 0) {
```

```

2 PTE *ptab = (PTE *)palloc_f();
3 memset(ptab, 0, PGSIZE);
4 pdir[pdir_idx] = (uintptr_t)ptab | PTE_P | PTE_W | PTE_U;
5 }

```

最后填写 PTE，使虚拟页映射到指定物理页：

```

1 PTE *ptab = (PTE *)PTE_ADDR(pdir[pdir_idx]);
2 ptab[ptab_idx] = PTE_ADDR(pa) | PTE_P | PTE_W | PTE_U;

```

loader() 的逻辑改为按页加载。如果 as 为 NULL，保留原来的直接加载逻辑；如果 as 非 NULL，就说明正在为用户进程准备地址空间，需要通过 _map() 建立映射：

```

1 uintptr_t va = (uintptr_t)DEFAULT_ENTRY;
2 size_t offset = 0;
3 while (offset < size) {
4     void *pa = new_page();
5     memset(pa, 0, PGSIZE);
6     _map(as, (void *) (va + offset), pa);
7
8     size_t len = size - offset;
9     if (len > PGSIZE) {
10         len = PGSIZE;
11     }
12     fs_read(fd, pa, len);
13     offset += PGSIZE;
14 }

```

这里每次循环完成三个动作：

第一，new_page() 分配一个空闲物理页。

第二，_map() 把用户程序的当前虚拟页映射到这个物理页。

第三，fs_read() 把程序文件中对应一页的内容读入物理页。

main() 中启用 HAS_PTE，并通过 load_prog() 加载默认程序：

```

1 #define HAS_PTE
2
3 #ifdef HAS_PTE
4     init_mm();
5 #endif
6
7 #ifdef HAS_PTE
8     load_prog(DEFAULT_PROGRAM);
9 #else
10     uint32_t entry = loader(NULL, DEFAULT_PROGRAM);
11     ((void (*)(void))entry)();
12 #endif

```

2.2.3 代码解释说明

`_protect()` 创建用户地址空间时会复制内核映射，因此切换到用户进程页目录后，内核代码和内核数据仍然可以被访问。`_map()` 只负责补充用户程序自己的虚拟页映射。

`_map()` 中 PDE 和 PTE 都设置了 `PTE_P`、`PTE_W`、`PTE_U`。`PTE_P` 表示映射有效，`PTE_W` 表示页面可写，`PTE_U` 表示用户页面。虽然当前 NEMU 没有实现 ring 3 权限检查，但保留这些标志符合 i386 页表语义，也方便后续阶段扩展。

`loader()` 不能直接把文件读到 `0x8048000`，因为这个地址是用户进程的虚拟地址，不一定存在于当前内核地址空间中。内核真正能直接访问的是 `new_page()` 返回的物理页对应地址。因此正确做法是先分配物理页，把虚拟页映射到物理页，再把文件内容读入该物理页。

若按本阶段第二节单独测试 dummy，需要把 `nanos-lite/src/main.c` 中默认程序临时设为：

```
1 #define DEFAULT_PROGRAM "/bin/dummy"
```

2.2.4 实现结果

实现正确后，看到 dummy 程序最后输出 GOOD TRAP 的信息，说明它确实在虚拟地址空间上成功运行了：

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:50:25, May 25 2026
For help, type "help"
(nemu) c
[src/mm.c,24,init_mm] free physical pages starting from 0x1d91000
[src/main.c,22,main] 'Hello World!' from Nanos-lite
[src/main.c,23,main] Build time: 05:56:27, May 25 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102f00, end = 0x1d4cea1, size = 29663137 bytes
[src/main.c,30,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032
```

2.3 在分页机制上运行仙剑奇侠传

2.3.1 实现思路

仙剑奇侠传运行过程中会使用 `malloc` 等动态内存分配函数，最终通过 `sbrk/brk` 系统调用扩大用户堆区。在没有分页机制时，`mm_brk()` 可以直接返回 0，因为 `0x4000000` 之后的物理内存可以被用户程序直接使用。开启分页后，用户程序看到的是虚拟地址空间，新申请的堆区虚拟页如果没有映射到物理页，就会在访问时触发 PTE/PDE present assertion。

因此，`mm_brk()` 需要在 `new_brk` 超过当前最大堆边界时，为新增堆区范围补充分配物理页并建立虚拟地址映射。为了简化，本阶段不实现堆区回收，`new_brk` 变小时只更新 `cur_brk`，不取消映射。

2.3.2 实现过程

首先在系统调用处理中把 `SYS_brk` 从直接返回 0 改为调用 `mm_brk()`：

```
1 case SYS_brk:
2     SYSCALL_ARG1(r) = mm_brk(a[1]);
3     break;
```

然后实现 `mm_brk()`。第一次调用时，初始化当前 `break` 和最大 `break`：


```

1 if (current->cur_brk == 0) {
2     current->cur_brk = current->max_brk = new_brk;
3 }

```

后续如果 new_brk 大于 max_brk，则说明堆区向高地址扩展，需要补充映射：

```

1 if (new_brk > current->max_brk) {
2     uintptr_t start = PGROUNDUP(current->max_brk);
3     uintptr_t end = PGROUNDUP(new_brk);
4     for (uintptr_t va = start; va < end; va += PGSIZE) {
5         void *pa = new_page();
6         memset(pa, 0, PGSIZE);
7         _map(&current->as, (void *)va, pa);
8     }
9     current->max_brk = new_brk;
10 }
11 current->cur_brk = new_brk;

```

其中 start 使用 PGROUNDUP(current->max_brk)，是为了避免 current->max_brk 位于某个已映射页面中间时重复映射该页面。end 使用 PGROUNDUP(new_brk)，保证覆盖 new_brk 之前所有可能被访问到的虚拟页。

main.c 中默认程序为仙剑奇侠传：

```

1 #define DEFAULT_PROGRAM "/bin/pal"

```

这样 Nanos-lite 初始化分页机制后，会通过 load_prog() 将 pal 加载到用户虚拟地址空间中，并在运行过程中由 mm_brk() 按需补充堆区映射。

2.3.3 代码解释说明

current 指向当前运行进程的 PCB。PCB 中保存了该进程的地址空间 as、当前堆顶 cur_brk 和已映射的最大堆边界 max_brk：

```

1 _Protect as;
2 uintptr_t cur_brk;
3 uintptr_t max_brk;

```

cur_brk 表示用户程序当前认为的 program break，max_brk 表示内核已经为该进程映射到的最高堆边界。由于本实验不释放页面，max_brk 只会增加。这样当用户程序反复增大和缩小堆区时，只有超过 max_brk 的新增范围才需要分配物理页。

mm_brk() 使用 _map(¤t->as, va, pa)，说明新增堆页被映射到当前进程自己的虚拟地址空间，而不是内核页表或其他进程页表。这样用户程序访问新堆区虚拟地址时，NEMU 的 page_translate() 能通过当前 CR3 找到正确的 PDE/PTE，并最终访问到 new_page() 分配出的物理页。

总结来说，本阶段形成了完整的分页运行路径：

```

1 Nanos-lite init_mm()
2     -> AM_pte_init()
3     -> NEMU 设置 CR3 和 CRO.PG

```

```

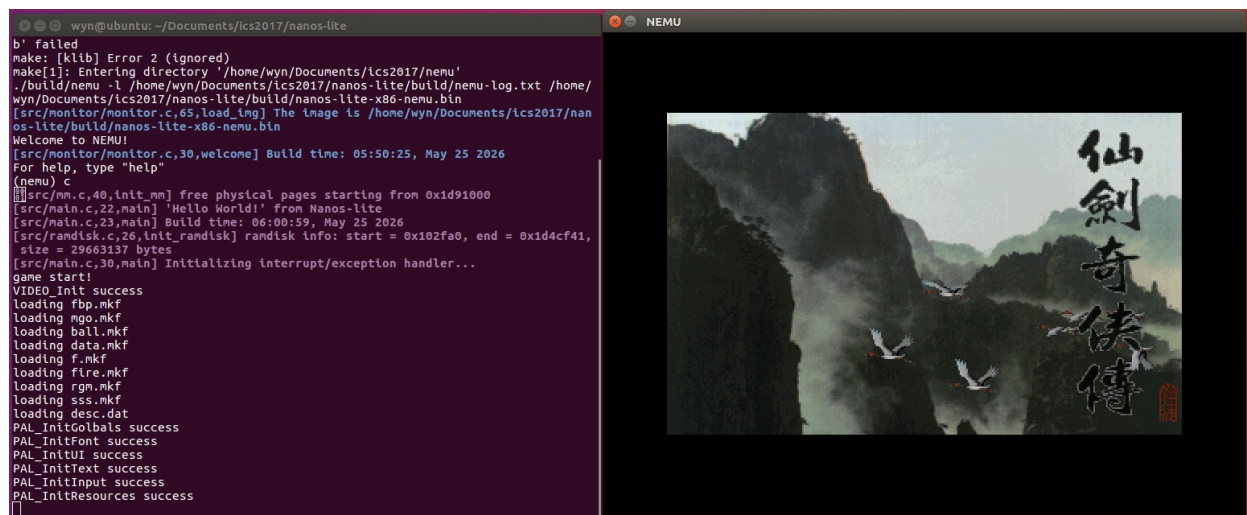
4 -> Nanos-lite load_prog()
5 -> AM _protect() 创建用户地址空间
6 -> loader() 分页加载用户程序
7 -> AM _map() 建立虚拟页到物理页映射
8 -> NEMU vaddr_read/vaddr_write()
9 -> page_translate() 完成地址转换
10 -> 用户程序在虚拟地址空间中运行

```

在此基础上, dummy 可以验证用户程序代码确实运行在独立虚拟地址空间上; 仙剑奇侠传可以进一步验证分页机制下的文件访问、设备访问以及堆区动态扩展。

2.3.4 实现结果

make update 后运行:



3 PA4-2

本阶段在第一阶段分页机制的基础上, 实现了从内核自陷进入调度器、通过陷阱帧完成上下文切换, 以及让仙剑奇侠传和 hello 程序分时运行。第一阶段已经可以把用户程序加载到独立虚拟地址空间中; 第二阶段的核心目标是: 不再由内核直接函数调用用户程序, 而是让内核通过异常返回路径恢复用户进程现场, 从而真正以“进程上下文切换”的方式运行用户程序。

3.1 实现内核自陷

3.1.1 实现思路

在第一阶段中, load_prog() 创建用户地址空间并加载用户程序后, 仍然通过函数调用的方式直接跳到用户程序执行。这种方式不符合真实操作系统的运行方式。真实系统中, 内核通常通过一次异常或中断返回路径切换到用户进程。

本实验中, x86-nemu 的 AM 约定使用 int \$0x81 作为内核自陷。内核自陷的作用是主动触发一次异常处理流程, 让 asm_trap 构造陷阱帧并调用 irq_handle()。irq_handle() 识别到 irq 号为 0x81

后，将其包装成 `_EVENT_TRAP` 事件交给 Nanos-lite。Nanos-lite 收到 `_EVENT_TRAP` 后调用 `schedule()`，返回要运行的用户进程陷阱帧。

因此，本要求需要完成以下内容：

第一，在 ASYE 中添加 `int $0x81` 对应的入口。

第二，实现 `_trap()`，让它执行 `int $0x81`。

第三，让 `irq_handle()` 把 `0x81` 转换为 `_EVENT_TRAP`。

第四，Nanos-lite 收到 `_EVENT_TRAP` 后进入调度。

第五，`main()` 加载用户程序后调用 `_trap()`，而不是直接调用用户程序入口。

3.1.2 实现过程

首先，在 `trap.S` 中添加内核自陷入口 `vectrap`。它和系统调用入口 `vecsyst` 的结构相同，先压入统一格式的 `error code` 和 `irq id`，然后跳转到 `asm_trap`：

```
1 .globl vecsys;    vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
2 .globl vectrap;  vectrap: pushl $0;  pushl $0x81; jmp asm_trap
3 .globl vecnull;  vecnull: pushl $0;  pushl $-1;  jmp asm_trap
```

这里 `0x80` 表示系统调用，`0x81` 表示内核自陷，`-1` 表示其它未处理异常。这样进入 `asm_trap` 后，栈上的陷阱帧格式始终统一。

然后，在 ASYE 初始化 IDT 时注册 `0x81` 号中断门：

```
1 idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
2 idt[0x81] = GATE(STS_TG32, KSEL(SEG_KCODE), vectrap, DPL_KERN);
```

系统调用 `0x80` 需要允许用户程序通过 `int $0x80` 进入，因此 `DPL` 设置为 `DPL_USER`。内核自陷 `0x81` 只由内核调用，因此 `DPL` 设置为 `DPL_KERN`。

接着实现 `_trap()`：

```
1 void _trap() {
2     asm volatile("int $0x81");
3 }
```

这样 Nanos-lite 调用 `_trap()` 时，会触发一次 `0x81` 异常，进入 AM 的异常处理流程。

`irq_handle()` 中增加 `0x81` 的事件识别：

```
1 switch (tf->irq) {
2     case 0x80: ev.event = _EVENT_SYSCALL; break;
3     case 0x81: ev.event = _EVENT_TRAP; break;
4     default: ev.event = _EVENT_ERROR; break;
5 }
```

Nanos-lite 的 `do_event()` 收到 `_EVENT_TRAP` 后输出日志并调用 `schedule()`：

```
1 case _EVENT_TRAP:
2     Log("Kernel trap: start user process scheduling");
3     return schedule(r);
```

最后, `main()` 中在加载程序后调用 `_trap()`:

```
1 #ifdef HAS_PTE
2     load_prog(DEFAULT_PROGRAM);
3     load_prog("/bin/hello");
4     _trap();
5 #else
6     uint32_t entry = loader(NULL, DEFAULT_PROGRAM);
7     ((void (*)(void))entry)();
8 #endif
```

`load_prog()` 中原来直接切换地址空间并调用用户程序入口的代码被注释掉:

```
1 // _switch(&pcb[i].as);
2 // current = &pcb[i];
3 // ((void (*)(void))entry)();
```

3.1.3 代码解释说明

内核自陷的本质是让内核主动进入异常处理流程。虽然 `_trap()` 是内核主动调用的函数,但它不会直接跳到用户程序,而是执行 `int $0x81`, 让 CPU 按异常处理机制保存 EIP、CS、EFLAGS, 并由 `trap.S` 继续保存通用寄存器。

`trap.S` 中的入口代码压入 `irq id` 和 `error code`, 是为了统一 `_RegSet` 的布局。之后 `asm_trap` 执行 `pushal`, 形成完整陷阱帧。Nanos-lite 并不直接操作汇编栈, 而是通过 `irq_handle()` 传入的 `_RegSet` 指针观察和修改进程现场。

`_EVENT_TRAP` 是第一次调度用户程序的触发点。内核初始化完成后, 调用 `_trap()`, Nanos-lite 收到 `_EVENT_TRAP`, 然后由 `schedule()` 选择第一个用户进程。最终 `asm_trap` 会使用 `schedule()` 返回的陷阱帧, 从异常返回路径进入用户程序。

3.1.4 实现结果

实现正确之后, 看到 Nanos-lite 触发了 `main()` 函数中最后的 `panic`:

```
[src/monitor/monitor.c,65,load_img] The image is /home/wyn/Documents/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:50:25, May 25 2026
For help, type "help"
(nemu) c
[src/mm.c,40,init_mm] free physical pages starting from 0x1d92000
[src/main.c,22,main] 'Hello World!' from Nanos-lite
[src/main.c,23,main] Build time: 06:40:49, May 25 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x103120, end = 0x1d4d0c1, size = 29663137 bytes
[src/main.c,30,main] Initializing interrupt/exception handler...
[src/irq.c,12,do_event] Kernel trap: start user process scheduling
[src/main.c,44,main] system panic: Should not reach here
nemu: HIT BAD TRAP at eip = 0x00100032
```

3.2 实现上下文切换

3.2.1 实现思路

上下文切换的本质是切换陷阱帧。用户程序触发异常或系统调用时，当前执行现场被保存在栈上的 `_RegSet` 中。如果在异常处理返回前，把栈顶指针切换到另一个进程的 `_RegSet`，再执行 `popal` 和 `iret`，就会恢复另一个进程的寄存器、EIP、CS、EFLAGS，从而完成进程切换。

为了实现这一机制，需要完成三部分：

第一，`_umake()` 为新进程人工构造初始陷阱帧，使它能被 `iret` 恢复执行。

第二，`schedule()` 保存当前进程陷阱帧，选择下一个进程，切换 CR3，并返回下一个进程的陷阱帧。

第三，`asm_trap` 使用 `irq_handle()` 的返回值更新 `esp`，使后续 `popal/iret` 恢复新进程现场。

3.2.2 实现过程

首先实现 `_umake()`。用户进程第一次被调度时并不是由真实异常产生的，因此内核需要在用户栈上人工构造一个能被 `asm_trap` 恢复的 `_RegSet`。

`_umake()` 先在用户栈顶预留 `_start()` 需要的参数栈帧。Navy-apps 的入口函数 `_start()` 形式为：

```
1 void _start(int argc, char *argv[], char *envp[])
```

当前实验暂不支持 `argv` 和 `envp`，因此把 `argc`、`argv`、`envp` 以及返回地址位置都置为 0：

```
1 uintptr_t *sp = (uintptr_t *)ustack.end;
2 *--sp = 0;
3 *--sp = 0;
4 *--sp = 0;
5 *--sp = 0;
```

然后在该栈帧下方构造 `_RegSet`：

```
1 _RegSet *tf = (_RegSet *)((uintptr_t)sp - sizeof(_RegSet));
2 memset(tf, 0, sizeof(_RegSet));
3 tf->eip = (uintptr_t)entry;
4 tf->cs = KSEL(SEG_KCODE);
5 tf->eflags = 0x2;
6 return tf;
```

其中 `eip` 设置为用户程序入口，`cs` 按指导书要求设置为 8，也就是 `KSEL(SEG_KCODE)`。`eflags` 设置为 `0x2`，保留 i386 EFLAGS 中恒为 1 的位。

`load_prog()` 加载程序后，将 `_umake()` 返回的陷阱帧记录到 PCB 中：

```
1 _Area stack;
2 stack.start = pcb[i].stack;
3 stack.end = stack.start + sizeof(pcb[i].stack);
4
5 pcb[i].tf = _umake(&pcb[i].as, stack, stack, (void *)entry, NULL, NULL);
```

然后实现 `schedule()`。调度器首先保存当前进程的陷阱帧：

```

1 if (current != NULL) {
2     current->tf = prev;
3 }

```

如果当前还没有运行进程，第一次调度选择 pcb[0]:

```

1 if (current == NULL) {
2     next = &pcb[0];
3 }

```

选择好下一个进程后，如果发生了真正的进程切换，就调用 __switch() 切换到该进程地址空间：

```

1 if (next != current) {
2     current = next;
3     __switch(&current->as);
4 }
5 return current->tf;

```

最后修改 asm_trap。原先 irq_handle() 返回后，汇编代码会继续使用当前 esp 恢复原进程。为了支持上下文切换，需要使用 irq_handle() 的返回值作为新的 esp：

```

1 pushl %esp
2 call irq_handle
3
4 movl %eax, %esp
5
6 popal
7 addl $8, %esp
8 iret

```

按照 x86 调用约定，C 函数返回值保存在 eax 中。因此 irq_handle() 返回的 __RegSet* 会出现在 eax，movl %eax, %esp 就把栈顶切换到新的陷阱帧。

3.2.3 代码解释说明

__RegSet 的布局必须和 trap.S 中 pushal、软件压入 irq/error_code、CPU 自动压入 eip/cs/eflags 的顺序一致。trap.S 恢复现场时先 popal，再跳过 irq 和 error_code，最后 iret 恢复 eip、cs、eflags：

```

1 popal
2 addl $8, %esp
3 iret

```

因此 __umake() 构造的 __RegSet 必须能被这三步正确消费。通用寄存器可以全部初始化为 0，因为用户程序第一次运行前不依赖这些寄存器的初值。eip 是最关键字段，它决定 iret 后从哪里开始执行。

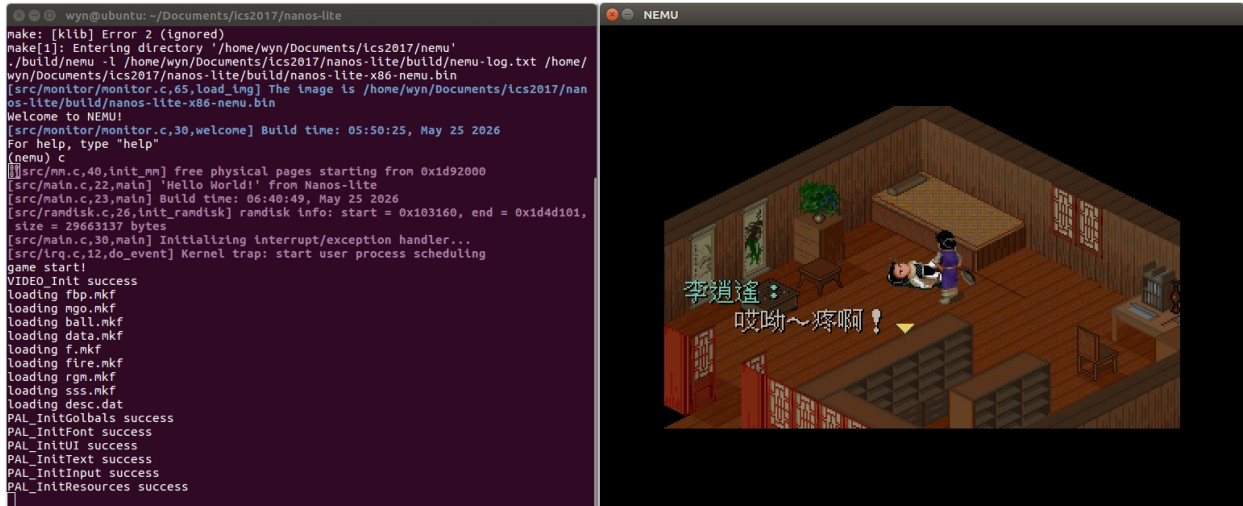
schedule() 只负责“选择谁运行”和“切换地址空间”，真正的寄存器恢复仍由 trap.S 完成。这样 AM 和 Nanos-lite 的分工比较清晰：Nanos-lite 决定调度策略，AM 负责底层上下文恢复。

__switch(¤t->as) 最终写 CR3，使 NEMU 的地址转换切换到目标进程页目录。因为每个用

户进程都有独立页表，所以同样的用户虚拟地址 0x8048000 可以映射到不同物理页。

3.2.4 实现结果

实现成功后, Nanos-lite 就可以通过内核自陷触发上下文切换的方式运行仙剑奇侠传了:



3.3 分时运行仙剑奇侠传和 hello 程序

3.3.1 实现思路

实现上下文切换后, Nanos-lite 可以加载多个用户程序。为了验证分时运行, 本阶段加载仙剑奇侠传和 hello 两个程序。由于当前阶段还没有实现时钟中断, 调度触发点选择在系统调用返回前。也就是说, 进程每次陷入内核处理系统调用后, Nanos-lite 可以选择是否切换到另一个进程。

仙剑奇侠传需要频繁访问文件、设备和 framebuffer, hello 程序则只是不断输出字符串。若二者严格 1:1 轮转, hello 会明显拖慢仙剑。因此调度策略调整为: 仙剑调度若干次后, 才让 hello 运行一次。这样既能看到 hello 输出, 又能保证仙剑获得更多运行机会。

另外, 分页开启后若 NEMU 每次虚拟地址访问都完整 page walk, 图形程序会非常慢。因此补充了一个简单 TLB, 用于缓存虚拟页到物理页的转换结果, 避免每次访存都访问两级页表。

3.3.2 实现过程

首先, main() 中加载两个用户程序:

```
1 load_prog(DEFAULT_PROGRAM);
2 load_prog("/bin/hello");
3 _trap();
```

其中 DEFAULT_PROGRAM 为:

```
1 #define DEFAULT_PROGRAM "/bin/pal"
```

接着, 在 do_event() 中让系统调用处理完成后进入调度:

```
1 case _EVENT_SYSCALL:
2     do_syscall(r);
```

```
3 return schedule(r);
```

这样每次用户程序通过 `int $0x80` 进入内核并处理完系统调用后，都会给调度器一次切换机会。调度策略中设置仙剑的调度频率：

```
1 #define PAL_TIME_SLICE 1000
```

`schedule()` 中使用静态计数器记录仙剑被调度的次数。`pcb[0]` 是仙剑，`pcb[1]` 是 `hello`。当当前进程是仙剑时，计数器递增；达到 `PAL_TIME_SLICE` 后，才切到 `hello` 一次。`hello` 下一次系统调用后切回仙剑：

```
1 static int pal_count = 0;
2 if (nr_proc == 1) {
3     next = &pcb[0];
4 }
5 else if (current == &pcb[0]) {
6     pal_count++;
7     if (pal_count >= PAL_TIME_SLICE) {
8         pal_count = 0;
9         next = &pcb[1];
10    }
11 }
12 else {
13     next = &pcb[0];
14 }
```

为了避免仙剑连续运行时重复写 `CR3`, `schedule()` 只有在 `next` 和 `current` 不同时才调用 `__switch()`：

```
1 if (next != current) {
2     current = next;
3     __switch(&current->as);
4 }
5 return current->tf;
```

最后，在 `NEMU` 中加入 `TLB` 优化。`TLB` 项保存虚拟页、物理页、`PTE` 地址和 `dirty` 状态：

```
1 typedef struct {
2     bool valid;
3     uint32_t tag;
4     uint32_t ppage;
5     paddr_t pte_addr;
6     bool dirty;
7 } TLBEntry;
```

`TLB` 命中时直接返回物理地址：

```
1 uint32_t vpage = addr & ~PAGE_MASK;
2 uint32_t page_off = addr & PAGE_MASK;
3 uint32_t idx = (addr >> 12) % TLB_SIZE;
4 if (tlb[idx].valid && tlb[idx].tag == vpage) {
```



```

5  if (write && !tlb[idx].dirty) {
6      PTE pte;
7      pte.val = paddr_read(tlb[idx].pte_addr, 4);
8      pte.dirty = 1;
9      paddr_write(tlb[idx].pte_addr, 4, pte.val);
10     tlb[idx].dirty = true;
11 }
12 return tlb[idx].ppage | page_off;
13 }

```

TLB miss 时完整 page walk，并把结果填入 TLB:

```

1  tlb[idx].valid = true;
2  tlb[idx].tag = vpage;
3  tlb[idx].ppage = pte.page_frame << 12;
4  tlb[idx].pte_addr = pte_addr;
5  tlb[idx].dirty = pte.dirty;
6  return tlb[idx].ppage | page_off;

```

写 CR3 时刷新 TLB:

```

1  case 3:
2      cpu.cr3.val = id_src->val;
3      tlb_flush();
4      break;

```

3.3.3 代码解释说明

当前阶段的分时运行是协作式调度，不是真正基于时钟中断的抢占式调度。进程只有在系统调用进入内核后，Nanos-lite 才有机会调用 `schedule()`。如果某个进程长时间不发生系统调用，它就不会被当前阶段的调度器抢占。

hello 程序会周期性输出字符串，因此能通过 `write/printf` 触发系统调用并回到内核。仙剑奇侠传在加载资源、读取设备、刷新屏幕时也会频繁触发系统调用。因此在本阶段，以系统调用返回点作为调度时机可以实现基本的分时运行。

PAL_TIME_SLICE 的含义不是 CPU 指令数，也不是时间片，而是仙剑经历的系统调用调度点数量。仙剑达到指定调度次数后，调度器才让 hello 运行一次。这样可以减少 hello 对仙剑图形运行的干扰。

TLB 的作用是缓存 `page_translate()` 的结果。没有 TLB 时，每次虚拟地址访问都要读取 PDE 和 PTE，这对仙剑这种大量访存和刷新画面的程序影响很大。加入 TLB 后，大多数同页访问可以直接得到物理页地址，只有 TLB miss 或 CR3 切换后才需要重新 page walk。

写 CR3 时必须刷新 TLB，因为不同进程的同一虚拟页可能映射到不同物理页。如果切换地址空间后继续使用旧 TLB 项，就会把一个进程的虚拟地址错误翻译到另一个进程的物理页。

本阶段最终形成的运行路径如下：

```

1  main()
2      -> load_prog("/bin/pal")
3      -> load_prog("/bin/hello")

```

```

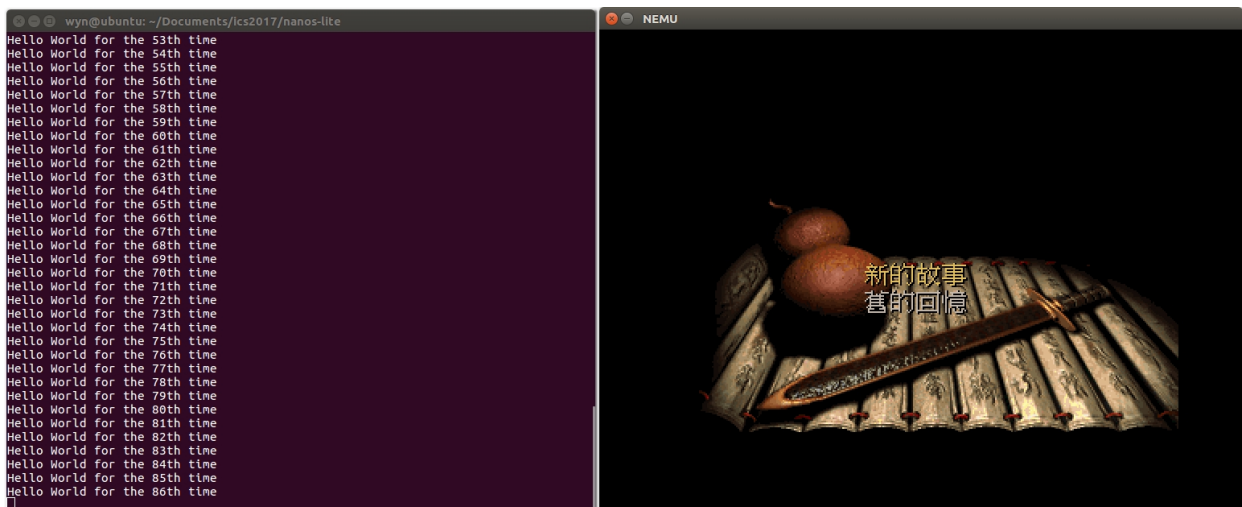
4  -> _trap()
5  -> int $0x81
6  -> asm_trap
7  -> irq_handle()
8  -> Nanos-lite do_event(_EVENT_TRAP)
9  -> schedule()
10 -> 返回目标进程 _RegSet
11 -> asm_trap 切换 esp
12 -> popal + iret
13 -> 用户进程运行
14 -> 用户进程系统调用
15 -> do_syscall()
16 -> schedule()
17 -> 在 pal 和 hello 之间分时切换

```

通过这一流程, Nanos-lite 不再直接函数调用用户程序, 而是依靠异常返回机制恢复用户进程现场; 多个用户程序也能通过保存和恢复各自 PCB 中的 tf, 实现基本分时运行。

3.3.4 实现结果

实现正确后, 看到仙剑奇侠传一边运行的同时,hello 程序也在一边输出:



4 PA4-3

PA4 第三阶段实验报告

本阶段在第二阶段“通过系统调用返回点进行进程调度”的基础上, 加入时钟中断, 实现真正由硬件中断驱动的分时多任务。第二阶段的调度仍然依赖用户程序主动触发系统调用, 如果某个程序长时间不系统调用, 操作系统就无法重新获得控制权。第三阶段通过时钟中断解决这个问题: 每隔一段时间, NEMU 的定时器设备主动向 CPU 发出中断请求, CPU 响应中断后进入 AM/Nanos-lite 的异常处理流程, Nanos-lite 在 `_EVENT_IRQ_TIME` 事件中调用 `schedule()`, 从而实现与用户程序行为无关的进程调度。

4.1 添加时钟中断

4.1.1 实现思路

硬件中断的核心思想是：设备在需要 CPU 处理事件时主动通知 CPU。时钟中断则是定时器设备周期性通知 CPU，使操作系统能周期性获得控制权。这样，即使当前运行的用户程序没有主动进行系统调用，CPU 也会在时钟中断到来时陷入内核，操作系统可以在中断处理过程中进行进程调度。

本阶段需要在三层系统中协同实现时钟中断。

第一层是 NEMU。NEMU 需要模拟 CPU 的 INTR 引脚。timer_intr() 被定时器触发后调用 dev_raise_intr(), dev_raise_intr() 将 cpu.INTR 置为高电平。CPU 每执行完一条指令后检查 cpu.INTR 和 EFLAGS.IF。如果 INTR 为真且 IF 为 1，就触发 32 号时钟中断。raise_intr() 在保存 EFLAGS 后将 IF 清零，模拟进入中断处理后关中断的行为。

第二层是 AM/ASYE。AM 需要为 32 号中断注册 IDT 入口，在 trap.S 中添加 vectimer 入口，并在 irq_handle() 中将 irq 号 32 包装成 _EVENT_IRQ_TIME 事件。

第三层是 Nanos-lite。Nanos-lite 收到 _EVENT_IRQ_TIME 后调用 schedule(), 让时钟中断成为进程切换的触发点。系统调用处理完成后不再直接调用 schedule(), 因为调度现在由时钟中断驱动。此外，为了让用户进程运行时能够响应时钟中断，_umake() 构造初始用户现场时需要打开 EFLAGS.IF。

4.1.2 实现过程

首先，在 NEMU 的 CPU_state 中增加 INTR 引脚：

```
1 CR0 cr0;
2 CR3 cr3;
3 bool INTR;
```

INTR 用来表示当前是否有外部设备中断请求。它相当于 CPU 的中断请求输入引脚。

在 restart() 中初始化 INTR：

```
1 cpu.cr0.val = 0x60000011;
2 cpu.cr3.val = 0;
3 cpu.INTR = false;
4 cpu.eflags.val = 0x00000002;
```

这样 NEMU 启动时没有待处理的外部中断请求。

定时器设备已经在 nemu/src/device/timer.c 中周期性调用 timer_intr(), timer_intr() 内部会调用 dev_raise_intr()。因此需要实现 dev_raise_intr()：

```
1 void dev_raise_intr() {
2     cpu.INTR = true;
3 }
```

这表示定时器把 CPU 的 INTR 引脚拉高。

然后在 exec_wrapper() 中，每执行完一条指令后轮询 INTR：

```
1 #define TIMER_IRQ 32
2
3 update_eip();
```

```

4
5 if (cpu.INTR && cpu.eflags.IF) {
6     cpu.INTR = false;
7     raise_intr(TIMER_IRQ, cpu.eip);
8     update_eip();
9 }

```

这里 `TIMER_IRQ` 为 32，表示时钟中断号。只有当 `cpu.INTR` 为真且 `EFLAGS.IF` 为 1 时，CPU 才响应外部中断。如果 `IF` 为 0，说明当前处于关中断状态，CPU 不响应该中断。

`raise_intr()` 中，在保存 `EFLAGS` 后关闭 `IF`：

```

1 rtlreg_t val = cpu.eflags.val;
2 rtl_push(&val);
3 cpu.eflags.IF = 0;

```

保存旧 `EFLAGS` 是为了将来 `iret` 能恢复中断前的状态。随后把当前 `cpu.eflags.IF` 清零，是为了让处理器进入中断处理流程后暂时关中断，避免中断嵌套带来复杂性。

AM 层需要在 `trap.S` 中添加 32 号时钟中断入口：

```

1 .globl vecsys;    vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
2 .globl vectrap;  vectrap: pushl $0;  pushl $0x81; jmp asm_trap
3 .globl vectimer; vectimer: pushl $0;  pushl $32;  jmp asm_trap
4 .globl vecnull;  vecnull: pushl $0;  pushl $-1;  jmp asm_trap

```

`vectimer` 和系统调用、自陷入口保持相同的陷阱帧格式：压入 `error code` 和 `irq id` 后跳转到 `asm_trap`。

在 `ASYS` 初始化 `IDT` 时，为 32 号中断注册入口：

```

1 idt[32] = GATE(STS_TG32, KSEL(SEG_KCODE), vectimer, DPL_KERN);
2 idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
3 idt[0x81] = GATE(STS_TG32, KSEL(SEG_KCODE), vectrap, DPL_KERN);

```

时钟中断来自硬件设备，不需要用户态通过 `int` 指令主动触发，因此 `DPL` 设置为 `DPL_KERN`。

`irq_handle()` 中识别 `irq` 号 32，并包装成 `_EVENT_IRQ_TIME`：

```

1 switch (tf->irq) {
2     case 0x80: ev.event = _EVENT_SYSCALL; break;
3     case 0x81: ev.event = _EVENT_TRAP; break;
4     case 32:  ev.event = _EVENT_IRQ_TIME; break;
5     default:  ev.event = _EVENT_ERROR; break;
6 }

```

为了让用户进程运行时能响应时钟中断，`_umake()` 中构造初始 `EFLAGS` 时设置 `IF` 位：

```

1 tf->eip = (uintptr_t)entry;
2 tf->cs = KSEL(SEG_KCODE);
3 tf->eflags = 0x2 | FL_IF;

```

其中 0x2 是 i386 `EFLAGS` 中保留为 1 的位，`FL_IF` 是中断使能位。

最后修改 Nanos-lite 的事件处理逻辑。系统调用只处理系统调用本身，不再调度：

```
1 case _EVENT_SYSCALL:
2     return do_syscall(r);
```

收到 `_EVENT_IRQ_TIME` 后输出日志并调用 `schedule()`：

```
1 case _EVENT_IRQ_TIME:
2     Log("Timer interrupt");
3     return schedule(r);
```

这样调度触发点就从“用户程序主动系统调用”变成“定时器硬件中断”。

4.1.3 代码解释说明

NEMU 中的 `cpu.INTR` 模拟 CPU 的外部中断引脚。定时器设备调用 `dev_raise_intr()` 后，只是把 `INTR` 置为 `true`，并不会立即打断当前指令。CPU 响应外部中断的时机是在一条指令执行完成后。因此 `exec_wrapper()` 末尾轮询 `INTR`，符合“每执行完一条指令检查中断请求”的硬件行为。

`EFLAGS.IF` 控制 CPU 是否响应外部中断。只有 `IF` 为 1 时，CPU 才处理 `INTR`。`raise_intr()` 保存旧 `EFLAGS` 后清除 `IF`，表示进入中断处理时关中断。之后 `iret` 会从陷阱帧中恢复旧 `EFLAGS`，如果旧 `EFLAGS` 的 `IF` 为 1，用户程序恢复运行后又可以继续响应后续时钟中断。

时钟中断号使用 32。NEMU 在响应时钟中断时调用：

```
1 raise_intr(32, cpu.eip);
```

`raise_intr()` 会根据 `IDTR` 在 `IDT` 中查找 32 号门描述符，跳转到 `AM` 注册的 `vectimer`。`vectimer` 压入 `irq id 32` 后进入 `asm_trap`，因此 `irq_handle()` 能通过 `tf->irq` 判断这是时钟中断。

`AM` 的 `irq_handle()` 不直接进行调度，它只把 `ISA` 相关的 `irq` 号转换为 `AM` 抽象事件 `_EVENT_IRQ_TIME`。具体调度策略属于操作系统职责，由 `Nanos-lite` 的 `do_event()` 调用 `schedule()` 完成。

`Nanos-lite` 收到 `_EVENT_IRQ_TIME` 后调用 `schedule()`，保存当前进程上下文并切换到下一个进程的上下文。由于时钟中断与用户程序是否调用系统调用无关，所以即使用户程序陷入死循环，定时器仍能让 CPU 陷入内核，操作系统仍有机会切换到其它进程。

相比第二阶段，第三阶段的调度路径从：

```
1 用户程序主动系统调用
2     -> _EVENT_SYSCALL
3     -> do_syscall()
4     -> schedule()
```

变为：

```
1 定时器周期触发
2     -> dev_raise_intr()
3     -> cpu.INTR = true
4     -> 指令执行结束后 CPU 检查 INTR 和 IF
5     -> raise_intr(32, cpu.eip)
6     -> vectimer
7     -> asm_trap
```

```

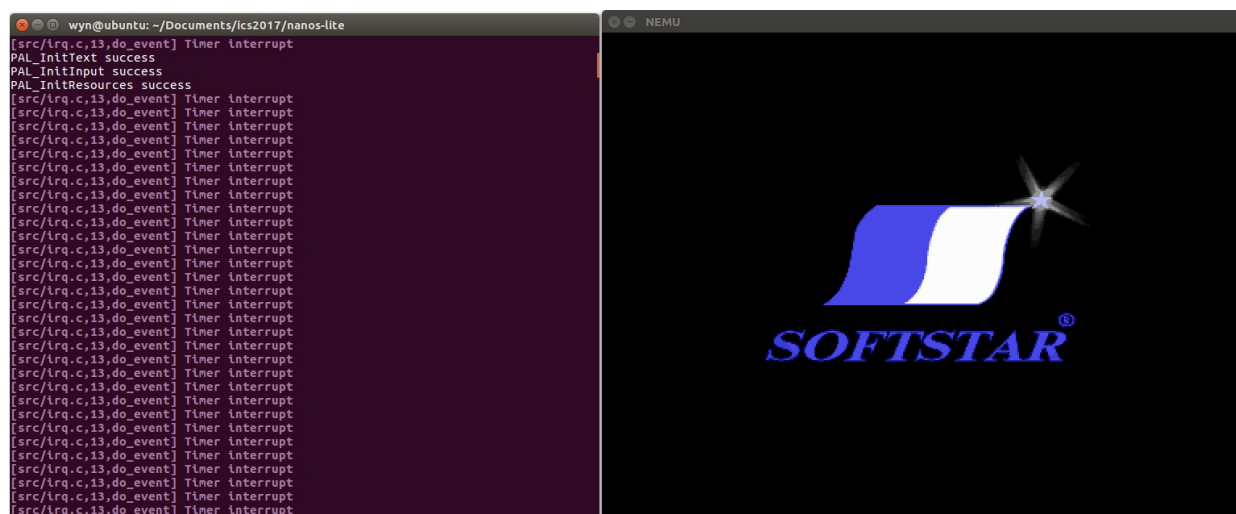
8  -> irq_handle()
9  -> _EVENT_IRQ_TIME
10 -> schedule()

```

这一变化使系统从协作式分时运行变成真正由硬件中断驱动的抢占式分时运行。

最终, 分页机制和时钟中断共同支撑了多个用户程序的运行: 分页机制为每个进程提供独立虚拟地址空间; 时钟中断周期性把控制权交还给操作系统; `schedule()` 保存和恢复不同进程的陷阱帧; `_switch()` 切换页目录; `asm_trap` 最终通过 `popal` 和 `iret` 恢复被调度进程的执行现场。通过这些模块协作, 仙剑奇侠传和 `hello` 程序能够在 Nanos-lite、AM 和 NEMU 构成的计算机系统中分时运行。

4.1.4 实现结果



可见, Nanos-lite 收到了 `_EVENT_IRQ_TIME` 事件且输出了我们设置好的 `log()`

5 展示你的计算机系统

本部分是在 PA4 第三阶段已经实现分页机制、上下文切换和时钟中断调度之后, 对系统进行展示性扩展。目标是让 Nanos-lite 同时加载第三个用户程序 `/bin/videotest`, 并在读取键盘事件时检测 F12 按键。当用户按下 F12 后, 当前参与调度的游戏程序在仙剑奇侠传和 `videotest` 之间切换, 同时 `hello` 程序仍然继续参与分时运行, 用于证明系统支持多个用户程序和动态调度目标切换。

5.1 实现思路

在前一阶段中, Nanos-lite 已经可以加载仙剑奇侠传和 `hello` 两个用户程序, 并通过时钟中断触发 `schedule()`, 在两个进程之间进行分时调度。本要求需要加入第三个用户程序 `videotest`, 并通过 F12 切换当前运行的“游戏程序”。

为了实现这个功能, 需要明确三个进程的角色:

```

1 pcb[0] = /bin/pal
2 pcb[1] = /bin/hello
3 pcb[2] = /bin/videotest

```

其中 `hello` 始终作为用于观察分时运行的辅助进程。真正需要切换的是游戏进程：一开始 `current_game` 指向 `pcb[0]`，即仙剑奇侠传；按下 F12 后，`current_game` 切换到 `pcb[2]`，即 `videotest`；再次按下 F12 后，又切换回 `pcb[0]`。

因此，调度器不再固定地在 `pcb[0]` 和 `pcb[1]` 之间切换，而是在 `current_game` 和 `pcb[1]` 之间切换：

```
1 current_game <-> hello
```

这样可以保证无论当前游戏是 `pal` 还是 `videotest`，`hello` 都仍然能够参与分时运行。同时，`current_game` 的值可以由键盘事件动态修改。

实现上分为三步：

第一，在 `main()` 中加载 `/bin/videotest`。

第二，在 `proc.c` 中增加 `current_game`，并修改 `schedule()` 的调度目标。

第三，在 `device.c` 的 `events_read()` 中检测 F12 按下事件，并调用 `switch_game()` 切换 `current_game`。

5.2 实现过程

首先，在 `main()` 中加载第三个用户程序。原先只加载 `pal` 和 `hello`，现在增加 `videotest`：

```
1 load_prog(DEFAULT_PROGRAM);
2 load_prog("/bin/hello");
3 load_prog("/bin/videotest");
4 _trap();
```

`DEFAULT_PROGRAM` 仍然是：

```
1 #define DEFAULT_PROGRAM "/bin/pal"
```

因此三个用户程序的加载顺序为：

```
1 第 1 次 load_prog(DEFAULT_PROGRAM) -> pcb[0] = /bin/pal
2 第 2 次 load_prog("/bin/hello")      -> pcb[1] = /bin/hello
3 第 3 次 load_prog("/bin/videotest") -> pcb[2] = /bin/videotest
```

`load_prog()` 每调用一次，就为对应程序创建一个 PCB，调用 `__protect()` 创建用户地址空间，调用 `loader()` 按页加载程序，并通过 `__umake()` 构造初始用户现场。

接着，在 `proc.c` 中增加 `current_game`：

```
1 static PCB *current_game = NULL;
```

`current_game` 用来记录当前应该参与调度的游戏进程。第一次进入 `schedule()` 时，还没有当前进程，因此将 `current_game` 初始化为 `pcb[0]`：

```
1 if (current == NULL) {
2     current_game = &pcb[0];
3     next = current_game;
4 }
```

这样系统一开始会运行仙剑奇侠传。

原先调度器固定在 pal 和 hello 之间切换。修改后，调度器在 current_game 和 hello 之间切换。为了减少 hello 对游戏程序的干扰，仍然保留游戏程序运行若干次后才切换到 hello 一次的策略：

```

1 static int game_count = 0;
2 if (nr_proc == 1) {
3     next = current_game;
4 }
5 else if (current == current_game) {
6     game_count++;
7     if (game_count >= PAL_TIME_SLICE) {
8         game_count = 0;
9         next = &pcb[1];
10    }
11 }
12 else {
13     next = current_game;
14 }

```

这里 pcb[1] 是 hello。也就是说，如果当前运行的是游戏程序，则累计游戏调度次数；达到阈值后让 hello 运行一次。如果当前运行的是 hello，则下一次调度切回 current_game。

选择好 next 后，如果 next 和 current 不同，就切换当前进程和地址空间：

```

1 if (next != current) {
2     current = next;
3     _switch(&current->as);
4 }
5 return current->tf;

```

这样即使 current_game 在 pal 和 videotest 之间切换，schedule() 也能通过 current_game 找到当前应运行的游戏进程，并通过 __switch() 切换到该进程的页目录。

然后实现 switch_game()：

```

1 void switch_game(void) {
2     if (nr_proc < 3) {
3         return;
4     }
5
6     current_game = (current_game == &pcb[2] ? &pcb[0] : &pcb[2]);
7 }

```

如果还没有加载第三个进程，则直接返回。正常情况下，pcb[2] 是 videotest，因此该函数会在 pal 和 videotest 之间切换 current_game。

最后，在 device.c 的 events_read() 中检测 F12。events_read() 原本负责读取键盘事件，并把按键转换为 Navy-apps 使用的文本事件格式：

```

1 kd KEY
2 ku KEY

```

新增逻辑如下：


```

1 if (key != _KEY_NONE) {
2     bool down = (key & 0x8000) != 0;
3     key &= ~0x8000;
4     if (down && key == _KEY_F12) {
5         switch_game();
6     }
7     snprintf(event, sizeof(event), "%s %s\n", down ? "kd" : "ku", keyname[key]);
8 }

```

这里 `_read_key()` 返回值的高位表示按下或释放。down 为 true 表示按键按下。只有在 F12 被按下时才调用 `switch_game()`，释放 F12 时不切换，避免一次按键动作触发两次切换。

5.3 代码解释说明

这个功能的关键在于把“当前游戏程序”从固定的 `pcb[0]` 抽象成 `current_game`。原先调度逻辑可以理解为：

```

1 pal <-> hello

```

加入 `videotest` 后，如果仍然固定使用 `pcb[0]`，就无法在 `pal` 和 `videotest` 之间切换。因此引入 `current_game` 后，调度逻辑变为：

```

1 current_game <-> hello

```

`current_game` 可以指向 `pcb[0]`，也可以指向 `pcb[2]`。`schedule()` 不需要关心当前游戏到底是哪一个程序，只要返回 `current_game` 的陷阱帧即可。

`switch_game()` 只修改 `current_game`，不直接切换 `current`，也不直接调用 `_switch()`。这是因为真正的上下文切换应该统一发生在 `schedule()` 中。F12 事件发生时，可能当前正在运行 `pal`、`videotest` 或 `hello`。`events_read()` 只负责更新调度目标；等下一次时钟中断触发 `schedule()` 时，调度器会根据 `current_game` 选择正确的进程并切换地址空间。

这种设计保持了职责分离：

```

1 events_read():
2     负责识别 F12，并修改 current_game
3
4 schedule():
5     负责选择下一个进程、保存和恢复 tf、切换地址空间
6
7 trap.S:
8     负责根据 schedule() 返回的 tf 恢复进程现场

```

F12 的检测放在 `events_read()` 中，是因为用户程序通过 `/dev/events` 获取键盘事件时，最终会调用 Nanos-lite 的 `events_read()`。在这里可以直接拿到 AM 的 `_read_key()` 返回值，并判断具体按键。因此这是实现按键控制调度策略的合适位置。

完整流程可以概括为：

```

1 Nanos-lite main()

```

```

2  -> load_prog("/bin/pal")
3  -> load_prog("/bin/hello")
4  -> load_prog("/bin/videotest")
5  -> _trap()
6  -> 时钟中断触发 schedule()
7  -> current_game 初始为 pcb[0]
8  -> pal 和 hello 分时运行
9  -> 用户按下 F12
10 -> events_read() 检测到 _KEY_F12
11 -> switch_game() 将 current_game 切换到 pcb[2]
12 -> 后续 schedule() 在 videotest 和 hello 之间分时运行
13 -> 再次按下 F12
14 -> current_game 切回 pcb[0]
15 -> 后续 schedule() 回到 pal 和 hello 分时运行

```

通过该功能，系统可以展示以下能力：

第一，Nanos-lite 能同时加载三个用户程序。

第二，每个用户程序都有自己的 PCB 和虚拟地址空间。

第三，时钟中断驱动的调度器可以在多个进程之间切换。

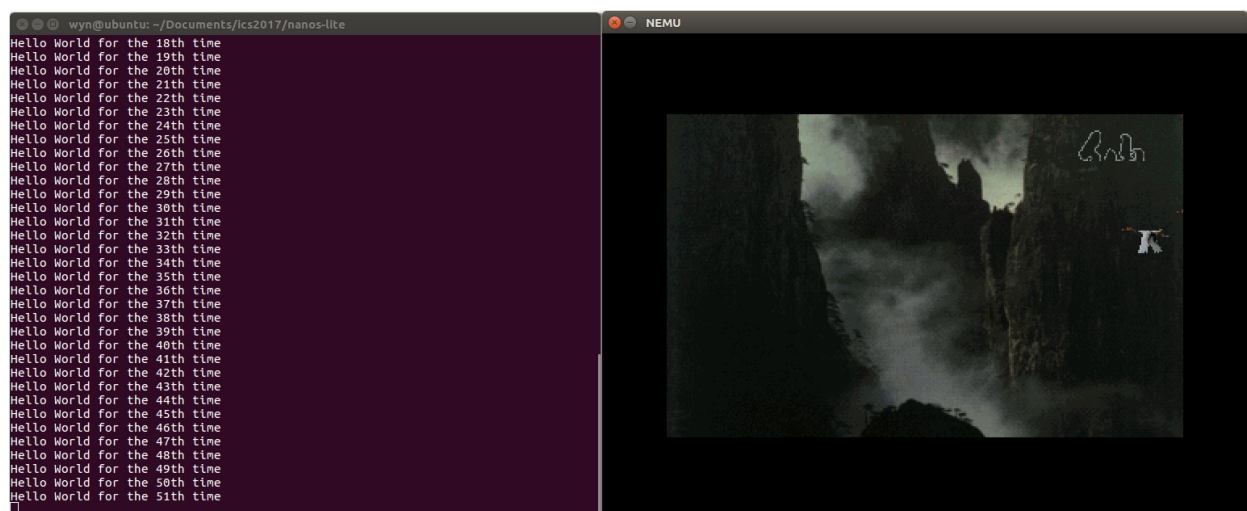
第四，设备输入可以影响操作系统调度策略。

第五，当前游戏程序可以在运行时动态切换，而 hello 仍然继续分时运行。

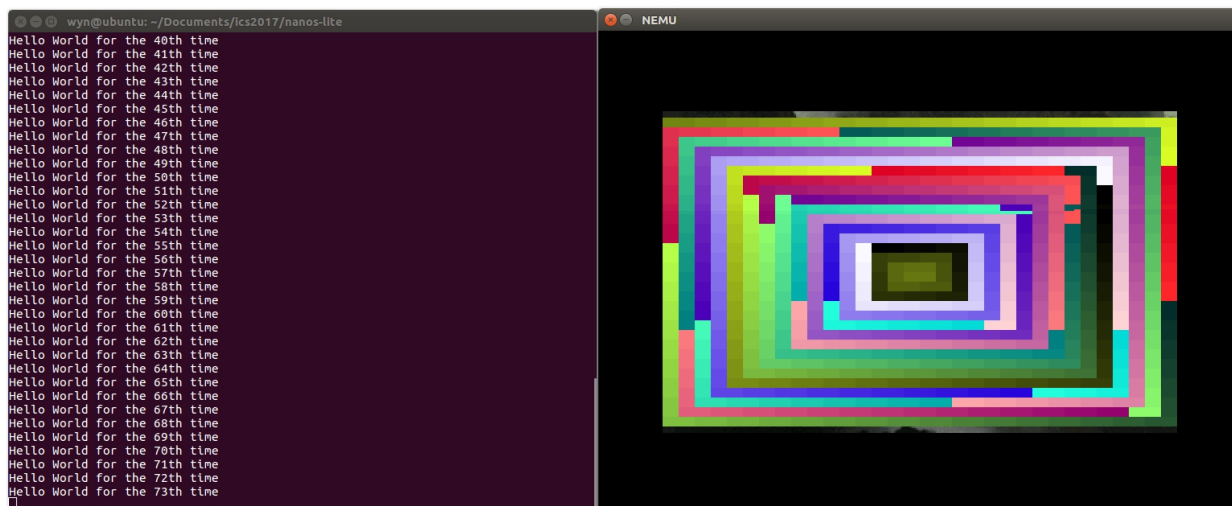
这说明前面实现的分页机制、上下文切换、时钟中断和设备事件处理已经能够协同工作，支撑一个简单但完整的多任务展示系统。

5.4 实现结果

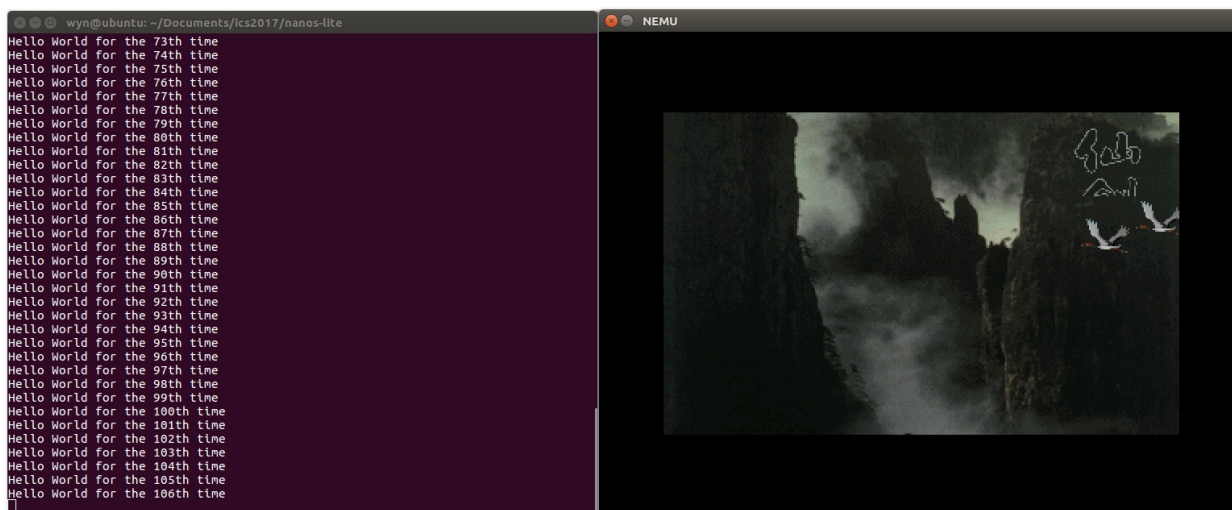
make run 开始运行：



可见，此时是仙剑与 hello 在分时运行，此时我们按下 F12：



可见，成功切换为了 videotest 和 hello 程序分时运行。此时我们再按下 F12:



可见又切回了仙剑，且仙剑是接着我们第一次切出时的状态继续运行的。

6 思考题与必答题

6.1 思考题

6.1.1 一些问题

问题 1: i386 不是一个 32 位的处理器吗，为什么表项中的基地址信息只有 20 位，而不是 32 位？

i386 是 32 位处理器，线性地址和物理地址在本实验中都使用 32 位表示。但分页机制以 4KB 为页面大小，页目录、页表和普通物理页都要求按 4KB 对齐。4KB 等于 2 的 12 次方，因此一个页框基地址的低 12 位必然全为 0。页表项没有必要保存这 12 个恒为 0 的位，只需要保存地址的高 20 位，也就是物理页框号。

换句话说，PDE/PTE 中的 20 位 page_frame 并不是完整物理地址，而是物理页框号。真正使用时，硬件会把它左移 12 位，再和线性地址的页内偏移拼接：

```

1 physical_page_base = page_frame << 12
2 physical_address = physical_page_base | page_offset

```

在本实验 NEMU 的 `page_translate()` 中也是这样做的：

```

1 uint32_t pdir_base = cpu.cr3.page_directory_base << 12;
2 paddr_t pte_addr = (pde.page_frame << 12) + ptab_idx * sizeof(PTE);
3 return (pte.page_frame << 12) | page_off;

```

因此，表项中只有 20 位基地址信息并不矛盾。因为页框天然 4KB 对齐，低 12 位可以由页内偏移提供，表项只需要保存高 20 位即可。

问题 2：手册上提到表项，包括 CR3，中的基地址都是物理地址，物理地址是必须的吗？能否使用虚拟地址？

CR3、PDE、PTE 中保存的基地址必须是物理地址。原因是地址转换过程本身就是为了把线性地址转换成物理地址。MMU 在进行 page walk 时，首先要根据 CR3 找到页目录，再根据 PDE 找到页表，再根据 PTE 找到物理页。如果 CR3 或表项中保存的是虚拟地址，那么 MMU 为了读取页目录或页表，又需要先对这些虚拟地址进行分页转换，这会导致递归依赖：

```

1 要转换 va
2   -> 需要读取 CR3 指向的页目录
3   -> 如果 CR3 是虚拟地址，就要先转换 CR3
4   -> 转换 CR3 又需要读取 CR3 指向的页目录
5   -> 形成循环

```

因此，硬件规定 CR3、PDE、PTE 中用于 page walk 的地址必须是物理地址。这样 MMU 可以直接访问物理内存中的页目录和页表，完成地址转换。

操作系统当然可以在自己的虚拟地址空间中为页表建立一个虚拟映射，方便内核通过普通指针修改页表。但这只是操作系统访问页表时使用的虚拟别名，写入 CR3/PDE/PTE 的值仍然必须是物理地址。

在本实验中，AM 的 `_map()` 把 `palloc_f()` 分配到的页表地址写入 PDE：

```

1 pdir[pdir_idx] = (uintptr_t)ptab | PTE_P | PTE_W | PTE_U;

```

而 Nanos-lite 的物理内存采用恒等映射，内核虚拟地址和物理地址数值相同，因此这里可以直接把指针值当作物理地址写入页表。这依赖于本实验的内核映射设计；从硬件语义上看，表项中保存的仍然是物理地址。

问题 3：为什么不采用一级页表？或者说采用一级页表会有什么缺点？

如果采用一级页表，需要为整个 4GB 线性地址空间中的每一个 4KB 页面准备一个页表项。4GB 地址空间中页面数量为：

```

1 4GB / 4KB = 2^32 / 2^12 = 2^20 = 1048576

```

每个页表项 4 字节，因此一个进程的一张一级页表大小为：

```

1 1048576 * 4B = 4MB

```

这意味着每创建一个进程，即使它只使用很小的一段虚拟地址空间，也需要分配完整的 4MB 页

表。对于大量小程序来说，这会造成严重内存浪费。

一级页表还有另一个问题：这 4MB 页表本身通常需要连续内存存放。连续大块内存更难分配，也更容易造成内存碎片。

i386 使用二级页表后，页目录只有 4KB，页表可以按需分配。一个进程只访问某些虚拟地址范围时，只需要为这些范围创建对应的二级页表：

```
1 一级页表：
2   每个进程固定需要完整 4MB 页表
3
4 二级页表：
5   先分配 4KB 页目录
6   只有访问或映射某个 4MB 虚拟区间时，才分配对应 4KB 页表
```

本实验中的 `__map()` 就体现了按需分配页表的思想：

```
1 if ((pdir[pdir_idx] & PTE_P) == 0) {
2     PTE *ptab = (PTE *)palloc_f();
3     memset(ptab, 0, PGSIZE);
4     pdir[pdir_idx] = (uintptr_t)ptab | PTE_P | PTE_W | PTE_U;
5 }
```

也就是说，只有当某个页目录项对应的页表不存在时，才通过 `palloc_f()` 申请一页物理内存作为页表。这样可以显著减少页表内存开销。

6.1.2 空指针真的是“空”的吗？

程序设计课上说 NULL 表示空指针，不指向任何东西。从语言语义上说，这是正确的：NULL 是一个特殊的指针值，用来表示“不指向有效对象”。但从计算机内部实现看，NULL 不是一种神秘的“空”，它通常就是数值 0，也就是虚拟地址 0。

当程序对空指针解引用时，例如：

```
1 int *p = NULL;
2 int x = *p;
```

CPU 并不知道这是 C 语言里的“空指针”。它只会把 `p` 的值当作地址使用。若 `p` 的值为 0，那么 CPU 会尝试访问线性地址 0：

```
1 load from virtual address 0x00000000
```

如果开启分页，MMU 会对虚拟地址 0 进行地址转换：

```
1 pdir_idx = 0x00000000 >> 22
2 ptab_idx = (0x00000000 >> 12) & 0x3ff
3 page_off = 0x00000000 & 0xfff
```

然后 MMU 检查对应 PDE 和 PTE。如果操作系统没有映射第 0 页，那么 `present` 位为 0，真实机器会触发 `page fault`。本实验 NEMU 中 `page_translate()` 会通过 `assertion` 检查 `present` 位：

```
1 Assert(pde.present, "invalid PDE for vaddr = 0x%08x", addr);
```

```
2 Assert(pte.present, "invalid PTE for vaddr = 0x%08x", addr);
```

因此，在分页机制下，“空指针解引用会出错”并不是因为硬件认识 NULL，而是因为操作系统故意不映射虚拟地址 0 附近的页面，使访问地址 0 变成非法访问。

如果某个系统真的把虚拟地址 0 映射到了某个有效物理页，那么空指针解引用并不会天然失败，而是会实际访问那一页内存。这说明 NULL 的“空”主要是语言和操作系统共同赋予的约定，不是地址 0 在硬件上天然不可访问。

我对空指针本质的认识是：空指针是一个特殊的指针值，通常表现为虚拟地址 0。它“不指向任何有效对象”是程序语言层面的语义；它在运行时是否触发错误，取决于操作系统和 MMU 是否把这个虚拟地址设置为不可访问。

6.1.3 内核映射的作用

在 `__protect()` 函数中创建用户进程地址空间时，有如下代码：

```
1 for (int i = 0; i < NR_PDE; i++) {
2     updir[i] = kpdirs[i];
3 }
```

这段代码的作用是把内核页目录 `kpdirs` 中的内核映射复制到用户进程页目录 `updir` 中。这样，当 CPU 切换到用户进程页目录后，内核代码、内核数据、设备映射、异常处理代码等仍然在当前地址空间中可访问。

如果注释掉这段代码，用户页目录中只会包含 `loader()` 和 `__map()` 后来为用户程序建立的用户虚拟页映射，而没有内核映射。这样会出现严重问题。

原因是进入用户进程前，Nanos-lite 会调用 `__switch(¤t->as)`，把 CR3 设置为用户进程页目录：

```
1 void _switch(_Protect *p) {
2     set_cr3(p->ptr);
3 }
```

一旦 CR3 指向用户页目录，CPU 后续所有虚拟地址访问都会按照用户页目录翻译。虽然当前正在执行的是内核代码，取指、访存、栈访问也都要经过当前页目录。如果用户页目录里没有内核映射，那么 CPU 继续执行内核代码时，就可能找不到对应的 PDE/PTE。

具体会发生类似下面的过程：

```
1 _switch(&user_as)
2   -> CR3 = user page directory
3   -> CPU 继续执行内核代码
4   -> 取指地址属于内核映射范围
5   -> 用户页目录中没有该 PDE/PTE
6   -> page_translate() 检查 present 失败
7   -> NEMU assertion failure 或真实机器 page fault
```

即使已经进入用户程序，用户程序触发系统调用或时钟中断后，也会跳回 AM 和 Nanos-lite 的异常处理代码。如果用户页目录没有内核映射，中断入口、`trap.S`、`irq_handle()`、`do_event()`、`schedule()` 等内核代码同样无法访问。

因此，用户进程的页目录不仅要包含用户程序自己的虚拟页映射，还必须包含内核映射。本实验没有实现特权级隔离，也没有在陷入内核时切换到单独的内核页目录，所以“每个用户页目录都复制内核映射”是内核能够在任意进程地址空间中运行的前提。

6.1.4 灾难性的后果

题目假设硬件把中断信息固定保存在内存地址 0x1000，AM 也总是从这里开始构造 trap frame。如果发生中断嵌套，就会出现灾难性后果。

正常情况下，trap frame 应该保存在栈上。每发生一次中断，栈向低地址增长，新的 trap frame 会放在当前栈顶下方，不会覆盖之前的 trap frame：

```
1 第一次中断：
2   在栈上保存 trap frame A
3
4 第二次中断嵌套：
5   在更低的栈地址保存 trap frame B
6
7 返回时：
8   先恢复 B
9   再恢复 A
```

如果 trap frame 固定保存在 0x1000，那么第一次中断发生时，硬件和 AM 会把被中断程序 A 的现场保存到 0x1000 开始的位置：

```
1 0x1000:
2   A.edi
3   A.esi
4   A.ebp
5   A.esp
6   A.ebx
7   A.edx
8   A.ecx
9   A.eax
10  A.irq
11  A.error_code
12  A.eip
13  A.cs
14  A.eflags
```

如果在处理中断 A 的过程中又发生了第二次中断 B，由于硬件和 AM 仍然从 0x1000 开始构造 trap frame，B 的现场会覆盖 A 的现场：

```
1 0x1000:
2   B.edi 覆盖 A.edi
3   B.esi 覆盖 A.esi
4   ...
5   B.eip 覆盖 A.eip
6   B.cs 覆盖 A.cs
7   B.eflags 覆盖 A.eflags
```


结果是，第一次中断的现场丢失了。第二次中断处理完后，也许能恢复到 B 被打断的位置；但当系统试图从第一次中断返回时，它已经找不到 A 原来的寄存器和 EIP，只能使用被 B 覆盖后的内容。

灾难性后果包括：

- 1 1. 原进程的 EIP 被覆盖，iret 返回到错误地址
- 2 2. 原进程的 ESP 被覆盖，栈被破坏
- 3 3. 通用寄存器被覆盖，程序状态错误
- 4 4. EFLAGS 被覆盖，中断开关和条件标志错误
- 5 5. 可能返回到中断处理程序内部，造成死循环
- 6 6. 可能跳到非法地址，触发异常或 NEMU assertion
- 7 7. 可能继续运行但数据已损坏，表现为随机错误

这一灾难的表现形式可能是程序崩溃、死循环、不断触发异常、返回地址错误、栈损坏、输出混乱，甚至看似正常运行一段时间后突然出错。

因此，中断现场必须保存在栈上。栈天然支持嵌套：每次中断都在当前栈顶继续压入新的 trap frame；返回时按后进先出的顺序恢复现场。这正是中断处理依赖栈的重要原因。

6.2 必答题

题目：请结合代码，解释分页机制和硬件中断是如何支撑仙剑奇侠传和 hello 程序在我们的计算机系统 Nanos-lite、AM、NEMU 中分时运行的。

仙剑奇侠传和 hello 程序能够在本实验系统中分时运行，依赖两个核心机制：分页机制和硬件中断。

分页机制解决“多个用户程序如何拥有各自独立地址空间”的问题。硬件中断解决“操作系统如何周期性重新获得 CPU 控制权并切换进程”的问题。二者结合后，Nanos-lite 才能在 NEMU 上支撑多个用户程序分时运行。

6.2.1 分页机制如何支撑多个用户程序共存

Navy-apps 中的用户程序都链接到固定虚拟地址 0x8048000：

```
1 LDFLAGS += -Ttext 0x8048000
```

这意味着 pal 和 hello 都认为自己的代码从 0x8048000 附近开始执行。如果没有分页，它们会互相覆盖。分页机制允许两个程序使用相同的虚拟地址，但映射到不同的物理页。

Nanos-lite 加载程序时，通过 load_prog() 为每个程序创建独立地址空间：

```
1 _protect(&pcb[i].as);
2 uintptr_t entry = loader(&pcb[i].as, filename);
3 pcb[i].tf = _umake(&pcb[i].as, stack, stack, (void *)entry, NULL, NULL);
```

_protect() 分配新的页目录，并复制内核映射：

```
1 PDE *updir = (PDE*)(palloc_f());
2 p->ptr = updir;
3 for (int i = 0; i < NR_PDE; i++) {
4     updir[i] = kpdirs[i];
5 }
```

loader() 加载用户程序时, 不再直接把文件读到 0x8048000, 而是每一页分配物理页, 然后调用 __map() 建立虚拟页到物理页的映射:

```
1 while (offset < size) {
2     void *pa = new_page();
3     memset(pa, 0, PGSIZE);
4     _map(as, (void *) (va + offset), pa);
5     fs_read(fd, pa, len);
6     offset += PGSIZE;
7 }
```

__map() 负责填写页目录和页表:

```
1 if ((pdir[pdir_idx] & PTE_P) == 0) {
2     PTE *ptab = (PTE *)palloc_f();
3     memset(ptab, 0, PGSIZE);
4     pdir[pdir_idx] = (uintptr_t)ptab | PTE_P | PTE_W | PTE_U;
5 }
6
7 PTE *ptab = (PTE *)PTE_ADDR(pdir[pdir_idx]);
8 ptab[ptab_idx] = PTE_ADDR(pa) | PTE_P | PTE_W | PTE_U;
```

这样, pal 和 hello 虽然都从虚拟地址 0x8048000 开始, 但它们所在 PCB 中的 as.ptr 指向不同页目录。调度器切换进程时调用 __switch():

```
1 void _switch(_Protect *p) {
2     set_cr3(p->ptr);
3 }
```

set_cr3() 会让 NEMU 的 CR3 指向当前进程的页目录。于是同一个虚拟地址在不同进程中会被 page_translate() 翻译到不同物理地址:

```
1 current = pal:
2     CR3 = pal page directory
3     0x8048000 -> pal physical page
4
5 current = hello:
6     CR3 = hello page directory
7     0x8048000 -> hello physical page
```

NEMU 中的 vaddr_read() 和 vaddr_write() 在分页开启后调用 page_translate():

```
1 if ((addr & PAGE_MASK) + len <= PAGE_SIZE) {
2     return paddr_read(page_translate(addr, false), len);
3 }
```

page_translate() 根据 CR3、PDE、PTE 完成地址转换:

```
1 pdir_base = cpu.cr3.page_directory_base << 12
2 pde = memory[pdir_base + pdir_idx * sizeof(PDE)]
```

```

3 pte = memory[pde.page_frame + ptab_idx * sizeof(PTE)]
4 paddr = pte.page_frame + page_off

```

因此，分页机制保证了 pal 和 hello 的代码、数据、堆区互不覆盖。mm_brk() 还会在用户堆区增长时补充映射：

```

1 for (uintptr_t va = start; va < end; va += PGSIZE) {
2     void *pa = new_page();
3     memset(pa, 0, PGSIZE);
4     _map(&current->as, (void *)va, pa);
5 }

```

这使仙剑奇侠传运行时通过 malloc/sbrk 申请堆空间也能得到正确虚拟页映射。

6.2.2 上下文切换如何保存和恢复进程状态

每个 PCB 中保存进程的上下文指针 tf 和地址空间 as：

```

1 _RegSet *tf;
2 _Protect as;
3 uintptr_t cur_brk;
4 uintptr_t max_brk;

```

用户进程第一次运行前，__umake() 构造初始 trap frame：

```

1 tf->eip = (uintptr_t)entry;
2 tf->cs = KSEL(SEG_KCODE);
3 tf->eflags = 0x2 | FL_IF;

```

之后每次中断或异常发生，trap.S 会把当前寄存器保存到栈上，形成新的 _RegSet：

```

1 asm_trap:
2     pushal
3     pushl %esp
4     call irq_handle
5     movl %eax, %esp
6     popal
7     addl $8, %esp
8     iret

```

schedule() 保存当前进程 tf，选择下一个进程，切换地址空间，并返回下一个进程的 tf：

```

1 if (current != NULL) {
2     current->tf = prev;
3 }
4
5 if (next != current) {
6     current = next;
7     _switch(&current->as);
8 }

```

```
9 return current->tf;
```

irq_handle() 的返回值会放在 eax 中，trap.S 执行：

```
1 movl %eax, %esp
```

于是栈顶切换到新进程的 trap frame。接下来的 popal 和 iret 会恢复新进程的通用寄存器、EIP、CS、EFLAGS。这样，从中断返回后，CPU 就已经在运行另一个用户进程。

6.2.3 硬件中断如何触发分时调度

第二阶段中，调度依赖系统调用。如果 hello 或其它程序不主动系统调用，操作系统就无法重新获得控制权。第三阶段加入时钟中断后，NEMU 的定时器会周期性调用 timer_intr()，进而调用 dev_raise_intr()：

```
1 void dev_raise_intr() {
2     cpu.INTR = true;
3 }
```

cpu.INTR 模拟 CPU 的外部中断引脚。NEMU 每执行完一条指令后检查 INTR 和 IF：

```
1 if (cpu.INTR && cpu.eflags.IF) {
2     cpu.INTR = false;
3     raise_intr(32, cpu.eip);
4     update_eip();
5 }
```

如果 IF 为 1，CPU 响应 32 号时钟中断。raise_intr() 保存 EFLAGS、CS、EIP，并清除 IF：

```
1 rtlreg_t val = cpu.eflags.val;
2 rtl_push(&val);
3 cpu.eflags.IF = 0;
4 val = cpu.cs;
5 rtl_push(&val);
6 val = ret_addr;
7 rtl_push(&val);
```

AM 在 IDT 中注册 32 号中断入口：

```
1 idt[32] = GATE(STS_TG32, KSEL(SEG_KCODE), vectimer, DPL_KERN);
```

trap.S 的 vectimer 把 irq id 32 压栈：

```
1 .globl vectimer; vectimer: pushl $0; pushl $32; jmp asm_trap
```

irq_handle() 识别 32 号中断，并包装为 _EVENT_IRQ_TIME：

```
1 case 32:
2     ev.event = _EVENT_IRQ_TIME;
3     break;
```

Nanos-lite 的 `do_event()` 收到 `_EVENT_IRQ_TIME` 后调用 `schedule()`:

```
1 case _EVENT_IRQ_TIME:
2     Log("Timer interrupt");
3     return schedule(r);
```

这就使调度不再依赖用户程序是否主动系统调用。只要定时器中断到来，操作系统就能进入内核并决定是否切换进程。

6.2.4 pal 和 hello 如何分时运行

Nanos-lite 在 `main()` 中加载多个用户程序:

```
1 load_prog("/bin/pal");
2 load_prog("/bin/hello");
3 _trap();
```

加载后, `pcb[0]` 是 `pal`, `pcb[1]` 是 `hello`。初始 `_trap()` 触发第一次调度, `schedule()` 选择 `pcb[0]`, 即 `pal`。之后时钟中断周期性触发 `_EVENT_IRQ_TIME`, `schedule()` 在 `pal` 和 `hello` 之间切换。

为了减少 `hello` 对 `pal` 的影响, 调度器可以让 `pal` 获得更多运行机会:

```
1 if (current == current_game) {
2     game_count++;
3     if (game_count >= PAL_TIME_SLICE) {
4         game_count = 0;
5         next = &pcb[1];
6     }
7 }
8 else {
9     next = current_game;
10 }
```

其中 `current_game` 初始为 `pal`。这样 `pal` 多次被调度后, `hello` 才运行一次。`hello` 输出字符串用于证明它仍然在运行; `pal` 正常加载资源和刷新画面, 说明游戏进程也在运行。

整个分时运行路径可以概括为:

```
1 NEMU timer_intr()
2     -> dev_raise_intr()
3     -> cpu.INTR = true
4     -> exec_wrapper() 检查 INTR 和 IF
5     -> raise_intr(32, cpu.eip)
6     -> AM vectimer
7     -> asm_trap 保存现场
8     -> irq_handle() 生成 _EVENT_IRQ_TIME
9     -> Nanos-lite do_event()
10    -> schedule()
11    -> 保存当前进程 tf
12    -> 选择 pal 或 hello
13    -> _switch() 切换 CR3
14    -> 返回目标进程 tf
```

```
15 -> asm_trap 切换 esp
16 -> popal + iret
17 -> 目标进程继续执行
```

因此，分页机制负责“多个进程如何拥有独立地址空间”，硬件中断负责“操作系统如何周期性夺回 CPU 控制权”，上下文切换负责“如何保存和恢复不同进程执行状态”。这三者共同支撑了仙剑奇侠传和 hello 程序在 Nanos-lite、AM、NEMU 构成的计算机系统中分时运行。